SECURITY — EXPLAINED FROM SCRATCH

Who are you, and may you do that?

The complete security flow of ArchitectPoint in plain English — every lock between a person's keyboard and a row in the database, what each one actually stops, and what it does not.

COVERS	Browser → network → server → database
ASSUMES	No prior knowledge of JWT, BCrypt, CORS or Spring Security
CHECKPOINTS	Seven, in the order a request meets them
COMPANION TO	ArchitectPoint — Workflow & Architecture Guide
GENERATED	2 October 2026

— Contents

1 **The whole thing as an office building**
One analogy that every later section hangs off

2 **The seven checkpoints — master diagram**
Every lock between the keyboard and the disk, in order

3 **The most important line in this document**
The trust boundary, and why half the "security" is only politeness

4 **Checkpoint 1 — The browser**
Form validation, the route guard, and what localStorage really is

5 **Checkpoint 2 — The journey over the wire**
CORS, in plain English, and why it protects your users and not your data

6 **Checkpoint 3 — The wristband (what a JWT is)**
Three parts, one seal; readable by anyone, forgeable by nobody

7 **Checkpoint 4 — The bouncer**
"Who are you?" — JwtAuthenticationFilter, line by line

8 **Checkpoint 5 — The door list**
"May you go in there?" — SecurityConfig, and the 401/403 difference

9 **Checkpoint 6 — The application layer**
The one question nobody asks: "is this row actually yours?"

10 **Checkpoint 7 — The database**
BCrypt, the one-way street, and why the password never comes back

11 **The life of a password**
From the keyboard to the disk and why it can never return

12 **Four attacks in slow motion**
Where each one dies — and the one that does not

13 **The honest scorecard**
What is genuinely protected, what is not, ranked by severity

14 **Coverage audit**
Every security-touching file in the repo, and where it is explained here

1 The whole thing as an office building

Before any code: one picture to hold in your head. Every security mechanism in this project maps onto something in an ordinary office building, and the mapping is exact enough that you can reason about the code using the analogy alone.

IN THE BUILDING	IN ARCHITECTPOINT	THE JOB IT DOES
Applying for a staff pass at the front desk	`POST /api/register`	You give your details and a password. The password is shredded before filing — see checkpoint 7.
Reception checks your face against the file and hands you a wristband	`POST /api/login`	If your password matches, you get a sealed wristband that says who you are and which department you are in.
The wristband — tamper-proof, printed with your name and department, holographic seal	The **JWT** (token)	Carried on every later visit. Anyone can read what's printed on it; nobody can make a fake one.
The map on your phone, which greys out rooms you can't enter	`authGuard` in the browser	Keeps you from wandering into the wrong corridor. It is a convenience. It does not lock anything.
Your pocket, where the wristband lives between visits	`localStorage`	On your own computer. Nobody guards your pocket but you.
The bouncer at the entrance who inspects the seal and looks you up in the staff register	`JwtAuthenticationFilter`	Answers one question only: *who are you?* Never: may you do this?
The laminated list on each door — "Accounts staff only"	`SecurityConfig`	Answers the second question: *may someone from your department open this particular door?*
The clerk inside the room, who does the actual work	Controllers and services	Assumes you would not be standing there if you weren't allowed in. Does no checking of its own.
The vault where the files live	MySQL `appdb`	Only the clerk has the key. No visitor ever touches it directly.
The complaints desk that writes the polite refusal slip	`GlobalExceptionHandler`	Turns every refusal into the same tidy note, so nothing leaks about why.

The two questions, kept separate on purpose

Almost every security bug in a system like this comes from blurring two questions that must stay apart:

QUESTION 1 — AUTHENTICATION

"Who are you?"

Answered by the **bouncer**, from the wristband.
Failing it gets you **401 Unauthorized** — *I don't know who you are.*

QUESTION 2 — AUTHORISATION

"May you do that?"

Answered by the **door list**, from your department.
Failing it gets you **403 Forbidden** — *I know exactly who you are, and no.*

Hold on to this. Every confusing symptom later in the project resolves to "I mixed up question 1 with question 2". A 401 where you expected a 403 doesn't mean your door list is wrong — it means the bouncer never worked out who you were in the first place, so the door list was never consulted.

2 The seven checkpoints

Here is every lock, in the order a single click meets them. Read it top to bottom — that is the order things actually happen.

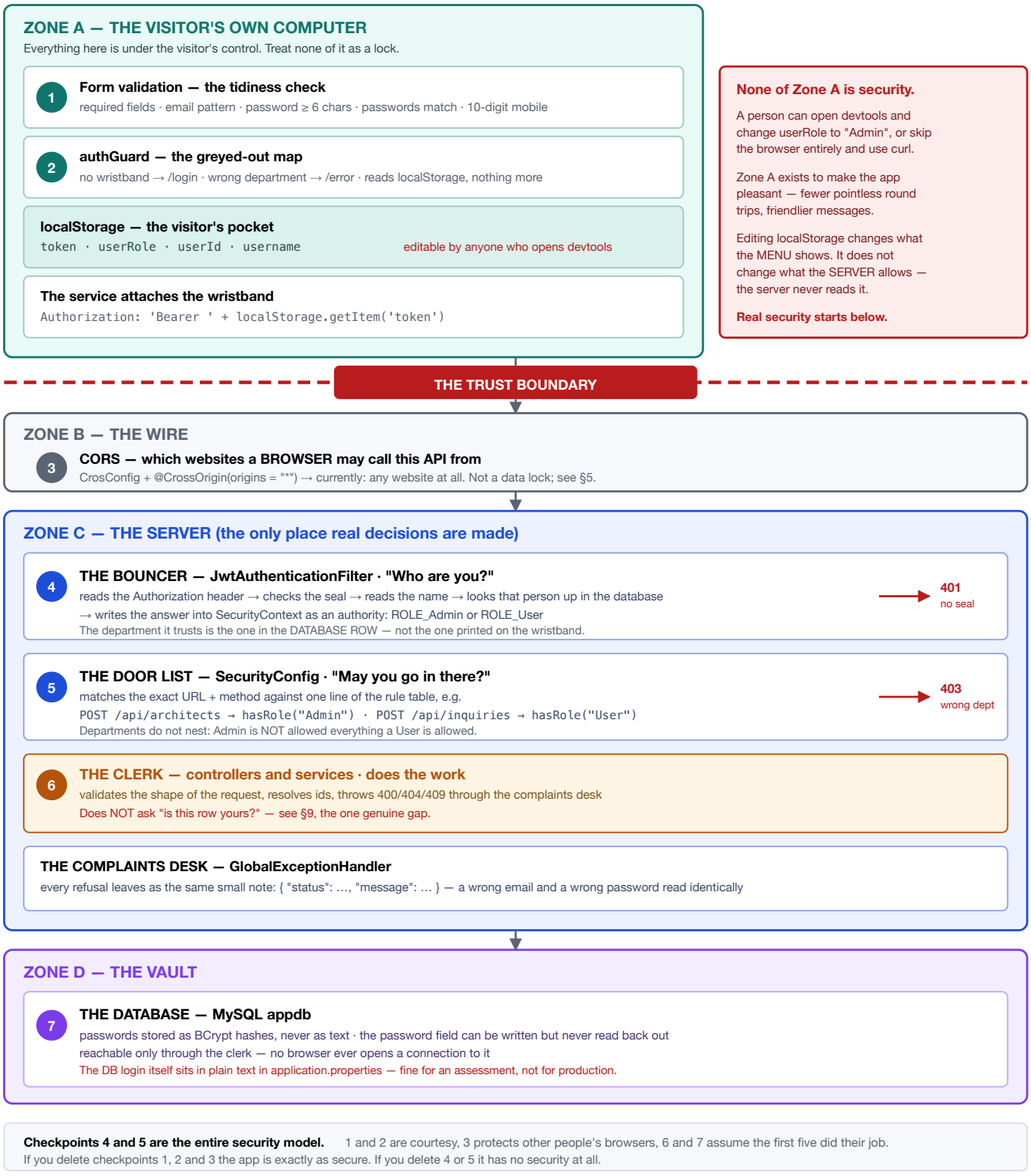


Figure 1 — The seven checkpoints. The thick red dashed line is the trust boundary: above it is the visitor's own machine and nothing there can be relied on; below it is your server, where every decision that matters is made.

3 The most important line in this document

If you remember one thing, remember the trust boundary. It is the difference between security that works and security that only looks like it does.

What the boundary means

Everything above the red line in Figure 1 — the Angular app, the form validation, the route guard, `localStorage` — runs **on the visitor's own computer**. They own that machine. They can:

- Open developer tools and change `localStorage.userRole` from `"User"` to `"Admin"`.
- Edit the running JavaScript to make `authGuard` always return `true`.
- Delete the `required` attributes from the form and submit empty fields.
- Ignore the browser altogether and send raw requests with `curl` or Postman.

All four take under a minute and need no special skill. So the honest way to describe the frontend is: **the Angular app is a convenient menu, not a lock.** It shows people the options they are likely to be allowed. The server decides what actually happens.

> **THE REASSURING HALF**
>
> Editing `localStorage.userRole` to `"Admin"` makes the admin menu appear in your browser — and then every admin button fails with **403**. You changed the menu, not the kitchen. The server never reads `localStorage`; it reads your wristband and then the database. Nothing you can type into your own browser changes either of those.

Why keep the frontend checks at all, then?

Three honest reasons, none of them "security":

1. **Fewer wasted trips.** Catching an empty password in the browser saves a round trip to a server that was only going to refuse it.
2. **Better messages.** "Password must be at least 6 characters", shown next to the field as you type, beats a generic 400 after you submit.
3. **No dead ends.** A User who types `/admin/architects` lands on a friendly error page instead of watching a half-drawn admin table fire a request that comes back 403.

The rule that follows from this

> **EVERY CHECK THAT MATTERS MUST EXIST ON THE SERVER — EVEN IF IT ALSO EXISTS IN THE BROWSER**
>
> Browser checks are a duplicate for comfort. Server checks are the real thing. If a rule exists *only* in Angular, it does not exist. This is why the project's second hard rule — keep Spring Security enabled, whatever the SRS says on p.43 — is not a grading quirk but the whole security model.

4 Checkpoint 1 & 2 — The browser

1 Form validation — the tidiness check

login.component.html · signup.component.html · Angular template-driven forms

Angular's `FormsModule` marks the form invalid and disables the submit button until every rule passes. In the signup form those rules are:

FIELD	RULE	WHAT IT IS REALLY FOR
Username	`required`	Stops an empty submission
Email	`required` + regex pattern	Catches "john@" before it costs a round trip
Password	`required`, `minlength="6"`	A floor on password length — **in the browser only**
Confirm password	must equal the password	Catches typos; checked again in `register()` before sending
Mobile	`pattern="[0-9]{10}"`	Shape only

NONE OF THESE RULES EXIST ON THE SERVER

There is no `@Valid`, no `@NotNull`, no `@Size` and no `unique = true` anywhere in the backend entities. A request sent with `curl` can register a user with a one-character password, an empty username, or an email that already exists. The browser's rules are advice; the server accepts whatever it is given. Worth knowing, and genuinely out of scope for the graded tests — but do not mistake the form rules for validation.

2 authGuard — the greyed-out map

guards/auth.guard.ts · applied by app-routing.module.ts

Every protected route carries a label saying which department it belongs to, and the guard reads that label before the screen is allowed to appear:

```
// app-routing.module.ts — the label
{ path: 'admin/architects', component: AdminViewArchitectComponent,
  canActivate: [authGuard], data: { role: 'Admin' } }

// guards/auth.guard.ts — the guard reading it
if (!authService.isLoggedIn())  return router.createUrlTree(['/login']);   // no wristband

const requiredRole = route.data?.['role'];
if (requiredRole === 'Admin' && !authService.isAdmin()) return router.createUrlTree(['/error']);
if (requiredRole === 'User'  && !authService.isUser())  return router.createUrlTree(['/error']);

return true;
```

In plain terms: *no wristband at all* → back to reception. *Wrong department* → the apology page. *Right department* → the screen loads. The whole decision is made from two values in `localStorage`, and no network request is involved.

WHAT THE GUARD GENUINELY ACHIEVES

- A logged-out visitor can never see an app screen
- A User typing an admin URL gets a clean error page, not a broken screen
- The right navbar renders — `app.component.html` picks the admin or user navbar from the same role value
- No pointless 403s from screens that were never going to work

WHAT IT DOES NOT ACHIEVE

- It cannot stop anyone — it runs on their machine
- Flipping `localStorage.userRole` walks straight past it
- It protects *screens*, never *data*
- Anyone skipping the browser never meets it at all

Where the wristband lives, and what logging out really does

```
// services/auth.service.ts — the only place identity is written
localStorage.setItem('token',    response.token);      // the wristband itself
localStorage.setItem('userRole', response.userRole);   // drives the guard + navbars
localStorage.setItem('userId',   String(response.userId));
localStorage.setItem('username', response.username);

// logout() — just four deletions
localStorage.removeItem('token'); // …and the other three
```

LOGGING OUT IS A LOCAL ACT

Logging out throws your wristband in the bin. It does not tell the building that the wristband is void. That token stays cryptographically valid for its **full 24 hours** — anyone who copied it beforehand can keep using it. There is no server-side revocation list, which is the normal trade-off for a stateless design: you buy "no session storage" and you pay with "cannot cancel a token early".

One more consequence of storing the token in `localStorage`: it is readable by any JavaScript running on the page. If an attacker ever got a script onto the page (an XSS flaw), they could read the token and walk off with the whole session. Nothing in this project currently creates such a hole — Angular escapes interpolated values by default — but it is why the choice between `localStorage` and an HttpOnly cookie is a real decision and not a style preference.

5 Checkpoint 3 — The journey over the wire

3 **CORS — which websites may call this API**
config/CrosConfig.java · @CrossOrigin(origins = "*") on all four controllers

CORS is the single most misunderstood piece of web security. It is worth getting right in your head, because it protects something quite different from what people assume.

What problem CORS solves

Browsers have a built-in rule: a page loaded from `evil.com` may not read the response from a request it makes to `yourbank.com`. Without that rule, any site you visited while logged into your bank could quietly read your balance using *your* logged-in browser.

CORS is how a server says "actually, it's fine, these particular sites may read my responses". Here, the server says it about every site:

```
// config/CrosConfig.java
registry.addMapping("/**")
        .allowedOrigins("*")                                          // any website
        .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
        .allowedHeaders("*");
```

Plus `@CrossOrigin(origins = "*")` on each controller. The practical reason is simple: the Angular app is served from port **8081** and the API lives on port **8080**. Different port means different origin, so without this the browser would block every single call and the app would not work at all.

WHAT CORS PROTECTS

- Other people's browsers, from scripts on other sites reading your API's responses using their logged-in session
- That's it. That is the entire job.

WHAT CORS DOES NOT PROTECT

- Your data — it is not a lock on the API
- Anything from `curl`, Postman or a script: those aren't browsers, so the rule simply doesn't apply to them
- Nothing about who you are — that is checkpoints 4 and 5

THE ONE-LINE SUMMARY

CORS is a rule *browsers* enforce on *other websites*. It is not a lock on your API. `allowedOrigins("*")` is wide open and completely normal for a development or assessment build; in production you would narrow it to the one domain your frontend is served from. Either way, it is checkpoints 4 and 5 that stop an attacker — not this.

The preflight, in one paragraph

For anything beyond a simple GET, the browser sends a small `OPTIONS` request first — "I'm about to send a POST with an Authorization header; is that allowed?" — and only sends the real request if the server says yes. That is why `OPTIONS` appears in the allowed methods list. If you ever see every call fail with `status: 0` and

nothing in the server log, a failed preflight is the usual cause: the browser gave up before the real request was ever sent.

What is *not* protecting the wire

> **EVERYTHING TRAVELS AS PLAIN HTTP**
>
> Both ports are `http://`, not `https://`. Passwords on their way to `/api/login` and tokens on every request afterwards cross the network unencrypted. On `localhost` that is harmless — the traffic never leaves your machine. On a real network it would mean anyone between the two machines could read a password or copy a wristband. TLS is the missing layer, and it is the one thing on this page that a deployment adds rather than code.

Related, and worth noticing: `application.properties` sets `spring.jpa.show-sql=true` and `logging.level.…BasicBinder=TRACE`, which prints every query *and its bound parameters* to the console. Handy while building. In production it writes user data into your log files.

6 Checkpoint 4a — The wristband: what a JWT actually is

A JWT is one long string that looks like gibberish. It is not encrypted, it is not random, and understanding its three parts removes most of the mystery from the rest of this document.

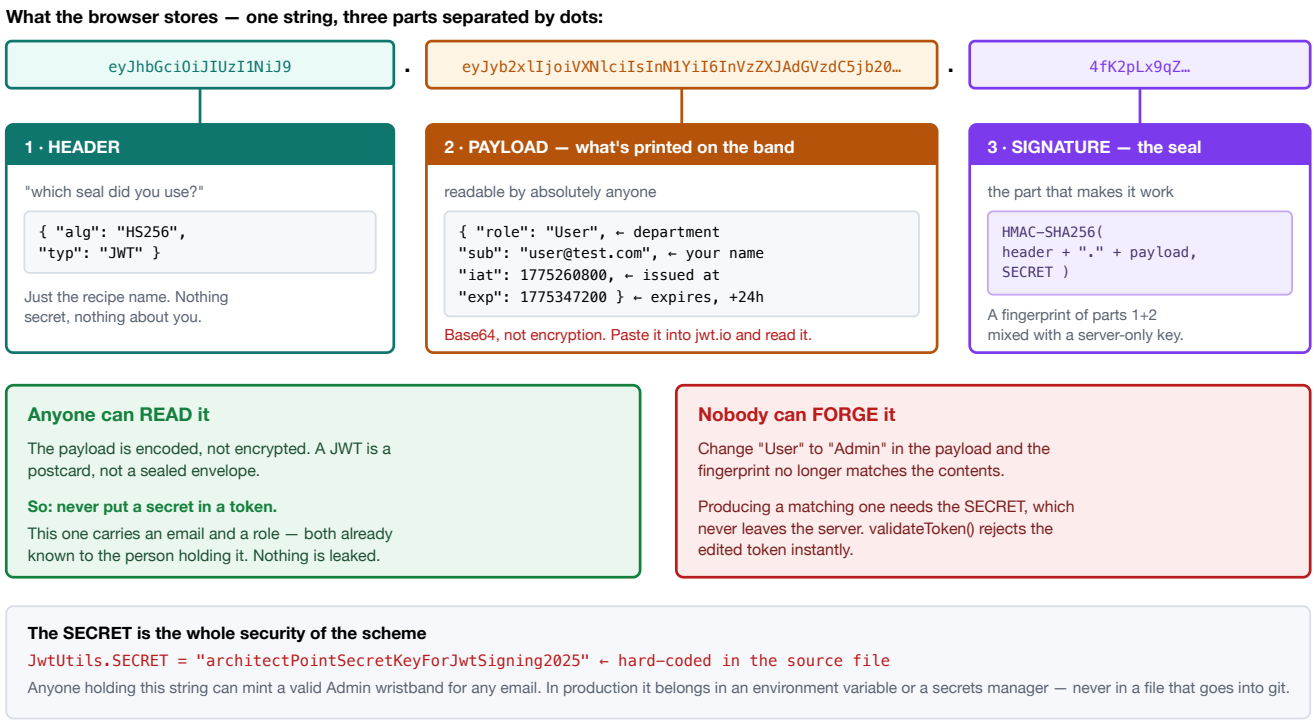


Figure 2 — Anatomy of the wristband. The seal is what makes a token trustworthy: it proves the contents have not changed since the server issued them. It proves nothing about who is currently holding the band.

Three consequences worth stating plainly

1. **A token is a bearer instrument.** Like a cinema ticket: whoever holds it gets in. The server has no way to tell the original owner from someone who copied it. That is why it should never travel over plain HTTP on a real network, and why the 24-hour expiry matters.
2. **Expiry is the only built-in brake.** `EXPIRATION_MS = 24 * 60 * 60 * 1000`. After a day the token stops working on its own; before then, nothing stops it. Shorter expiry means less exposure and more re-logins — that is the dial you turn.
3. **The signature says nothing about permission.** A perfectly valid, perfectly signed token is still refused by checkpoint 5 if it is the wrong department. Valid ≠ allowed. That is the whole reason 401 and 403 are different numbers.

> **ONE SUBTLETY THIS PROJECT GETS RIGHT**
>
> The role printed on the wristband is *not* what the server acts on. On every request the bouncer looks the person up in the database and uses the role on **that row**. So if an admin were demoted in the database, their existing token would lose admin powers on the very next request — no reissue needed. The token's `role` claim is effectively a convenience copy. (See §7 for exactly where this happens.)

7 Checkpoint 4b — The bouncer

4

JwtAuthenticationFilter — "Who are you?"

config/JwtAuthenticationFilter.java · runs before every other filter, on every request

This class runs on *every single request* to the API, before any of your code. Its only output is an answer to one question, written somewhere Spring can read it.

```
// config/JwtAuthenticationFilter.java — the whole method, annotated
String header = request.getHeader("Authorization");

if (header != null && header.startsWith("Bearer ")) {       // ① is a band being shown?
    String token = header.substring(7);                      //    strip the word "Bearer "
    try {
        if (jwtUtils.validateToken(token)) {                 // ② is the seal genuine + unexpired?
            String email = jwtUtils.extractUsername(token);  // ③ read the name off the band
            if (email != null && …getAuthentication() == null) {
                UserDetails userDetails =
                    userDetailsService.loadUserByUsername(email);   // ④ look them up in the
register
                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(auth);  // ⑤ pin on the badge
            }
        }
    } catch (Exception e) {
        SecurityContextHolder.clearContext();                // ⑥ anything odd → no identity, no
crash
    }
}
filterChain.doFilter(request, response);                     // ⑦ always continue
```

The seven steps in plain English

STEP	IN THE BUILDING	WHAT IT MEANS IN PRACTICE
①	"Are you wearing a band?"	No `Authorization` header → the filter does nothing and steps aside. Correct: `/login` and `/register` need no identity.
②	"Is the seal genuine, and in date?"	Re-computes the fingerprint with the secret and compares; also checks `exp`. An edited or expired token fails here.
③	"What name is printed on it?"	Reads the `sub` claim — the email.
④	"Look that person up in the register"	`MyUserDetailsService` runs `SELECT … WHERE email = ?`. One extra query per request — and the reason a deleted or demoted user loses access immediately.
⑤	"Pin a badge on them"	Writes the identity into `SecurityContextHolder`, where the door list will look for it. **This line is the entire point of the class.**
⑥	"Anything odd — no badge, no fuss"	A malformed token never throws out of the filter. It leaves the context empty, so you get a clean 401 instead of a 500 stack trace.
⑦	"Carry on either way"	The filter never rejects anything. It only reports. Rejecting is checkpoint 5's job.

Where the department actually comes from

Step ④ loads the database row; `UserPrinciple` then converts it into the one thing Spring Security understands — an *authority* string:

```
// config/UserPrinciple.java
public Collection<? extends GrantedAuthority> getAuthorities() {
    return List.of(new SimpleGrantedAuthority("ROLE_" + user.getUserRole()));
}
// userRole "Admin" in the DB  →  authority "ROLE_Admin"
// userRole "User"  in the DB  →  authority "ROLE_User"
```

> **WHY THE ROLE_ PREFIX EXISTS**
>
> When the door list says `hasRole("Admin")`, Spring silently looks for the authority `"ROLE_Admin"`. That prefix is a convention, and it has to be added by hand here. Build the authority without it and every rule fails — with a 403 that looks like a role problem but is really a spelling problem. This single line is why roles work at all.

The refusal slip when there is no identity

```
// config/JwtAuthenticationEntryPoint.java — the whole class
response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Unauthorized");   // 401
```

If a request reaches a protected endpoint with no identity pinned on, Spring calls this. It is the bouncer saying "I don't know who you are" — question 1 failed. Note what it does *not* say: nothing about which endpoint, which role was needed, or whether the user exists.

8 Checkpoint 5 — The door list

5

SecurityConfig — "May you go in there?"

config/SecurityConfig.java · the single source of truth for permissions

One block of code decides every permission in the entire application. There is no `@PreAuthorize` anywhere in the project — deliberately, because two sources of truth for permissions is how permission bugs happen.

The settings above the rules, and what each one means

LINE	IN PLAIN ENGLISH
`csrf(csrf -> csrf.disable())`	Turns off cross-site-request-forgery tokens. Safe *here* because this API authenticates with a header the browser never attaches automatically — unlike a cookie. CSRF attacks rely on the browser sending credentials by itself; a `Bearer` header only arrives if JavaScript puts it there.
`sessionCreationPolicy(STATELESS)`	"Never remember anyone between requests." No server session, no `JSESSIONID`. Every request must prove itself from scratch. This is what makes the wristband necessary.
`exceptionHandling(…EntryPoint)`	"When someone unidentified knocks, send the 401 slip."
`addFilterBefore(jwtAuthenticationFilter, …)`	"Put our bouncer at the door, ahead of Spring's own username/password filter." Order matters: the badge must be pinned on before the door list is read.

The rules themselves — read top to bottom, first match wins

```
.requestMatchers("/api/register", "/api/login").permitAll()   // reception: no band needed
.requestMatchers("/error").permitAll()                        // the complaints desk — see below

.requestMatchers(POST, "/api/architects").hasRole("Admin")    // add a listing → Admin only
.requestMatchers(GET,  "/api/architects").authenticated()     // read the list → ANY logged-in
person
.requestMatchers(GET,  "/api/architects/*").hasRole("Admin")  // read ONE listing → Admin only
.requestMatchers(PUT,  "/api/architects/*").hasRole("Admin")

.requestMatchers(GET,  "/api/inquiries/user/*").hasRole("User")  // my own inquiries → User only
.requestMatchers(POST, "/api/inquiries").hasRole("User")         // send an inquiry → User only
.requestMatchers(GET,  "/api/inquiries").hasRole("Admin")        // see everyone's → Admin only
.requestMatchers(PUT,  "/api/inquiries/*").hasRole("Admin")      // answer one → Admin only

.requestMatchers(GET,  "/api/feedback/user/*").hasRole("User")
.requestMatchers(POST, "/api/feedback").hasRole("User")
.requestMatchers(GET,  "/api/feedback").hasRole("Admin")

.anyRequest().authenticated()                                 // anything unlisted: at least be
logged in
```

The thing that surprises everyone: the departments do not nest

In most systems an admin can do everything a user can, plus more. **Not here.** Look at the pattern:

ACTION	ADMIN	USER	THE SENSE OF IT
Add an architect to the catalogue	allowed	403	Curating the catalogue is the admin's job
Browse the catalogue	allowed	allowed	The one shared door
Send an inquiry	403	allowed	An admin is not a customer — they have nobody to enquire to
Leave feedback	403	allowed	Same reasoning: feedback comes from customers
Read *everyone's* inquiries	allowed	403	Triage queue — admin's job
Read *my own* inquiries	403	allowed	"My" has no meaning for an admin, so the door is closed to them

So the two roles are not a ladder, they are two separate sets of keys that happen to share one door. Once that clicks, the matrix stops looking arbitrary: each role can do the things its job requires, and nothing else.

Three ways to write this block wrongly

1 · USING ** INSTEAD OF *

`/api/architects/*` means "exactly one more path segment" — it matches `/api/architects/7` but not `/api/architects`. Write `**` and it matches both, so the Admin-only "one listing" rule swallows the shared "browse the list" rule, and Users start getting 403 on the one endpoint that is supposed to be open to them.

2 · PUTTING THE BROAD RULE BEFORE THE NARROW ONE

First match wins. The specific `/api/inquiries/user/*` line must come before the general `/api/inquiries` line. Declaring narrow-before-broad keeps the file correct no matter what the wildcards do.

3 · FORGETTING `permitAll()` ON `/error`

This one is genuinely surprising. When a 403 is produced, the servlet container re-dispatches internally to `/error` to render the body — and that re-entry goes through the filter chain *again*. If `/error` needed authentication, the entry point would fire and overwrite the 403 with a **401**. Every "forbidden" test in the suite would then fail, pointing at role rules that were perfectly correct all along.

Reading the two numbers as a diagnostic

CODE	THE BUILDING SAYS	WHAT TO GO AND LOOK AT
401	"I don't know who you are."	The **bouncer**. Missing or malformed header, bad signature, expired token — or the filter isn't writing to `SecurityContext` at all.
403	"I know who you are. No."	The **door list**. Identity was established fine; the role doesn't match that line.

THE SINGLE MOST USEFUL DEBUGGING RULE IN THIS PROJECT

Getting **401 where you expected 403** almost never means your roles are wrong. It means authentication broke, so authorisation was never reached. Go and look at the filter, not the rule table. (Every "forbidden" test in the suite sends a perfectly valid token that simply carries the other role — so if those come back 401, the bouncer is the suspect.)

9 Checkpoint 6 — The application layer

6 **Controllers and services — the clerk**
4 controllers · 4 services · GlobalExceptionHandler

By the time your code runs, both security questions have been answered. The clerk's checks are about *correctness*, not permission — with one important exception that isn't checked at all.

What the clerk does check

CHECK	RESULT	WHERE
Does this inquiry name both a user and an architect?	throws	`ArchitectInquiryServiceImpl.createInquiry`
Do those ids actually exist?	throws	the same method — `findById(...).orElseThrow(...)`
Does the record you're updating exist?	404	`ResourceNotFoundException`
Do the login credentials match?	401	`UserServiceImpl.loginUser`

One nice touch worth copying

```
// service/UserServiceImpl.java
return userRepo.findByEmail(user.getEmail())
        .filter(found -> passwordEncoder.matches(user.getPassword(), found.getPassword()))
        .orElseThrow(() -> new InvalidCredentialsException("Invalid email or password."));
```

An unknown email and a wrong password take the same branch and produce the same sentence. That matters more than it looks: if "no such user" and "wrong password" read differently, anyone can feed a list of emails to the login endpoint and learn which ones have accounts — useful for a later attack. One message for both closes that off. The same discipline shows in `GlobalExceptionHandler`, which renders every failure as the same small envelope:

```
{ "status": 401, "message": "Invalid email or password." }
```

The one question nobody asks

THE SERVER CHECKS YOUR ROLE, NEVER YOUR OWNERSHIP

Consider `GET /api/inquiries/user/2`. The door list asks "are you a User?" — and that is the last question anyone asks. Nothing compares the `2` in the URL against the person actually holding the wristband. So a logged-in User can simply change the number:

```
GET /api/inquiries/user/3  →  200 OK, with somebody else's inquiries
```

The same applies to `/api/feedback/user/{id}`. In the UI it never happens, because the number comes from `localStorage.userId` — but as established in §3, the UI is not a lock.

This is called a **broken object-level authorisation** flaw, and it is the most common serious bug in real REST APIs. It is also entirely outside the 24 graded tests, which is why the code does not address it. Were you to close it, the fix is small and belongs in the service, not the config — compare the path variable with the authenticated principal:

```
// illustrative only — NOT a change to make for the assessment
UserPrinciple principal = (UserPrinciple) SecurityContextHolder
        .getContext().getAuthentication().getPrincipal();

if (!principal.getUser().getUserId().equals(userId)) {
    throw new AccessDeniedException("That isn't yours.");   // → 403
}
```

DO NOT ADD THIS TO THE PROJECT

The graded suite calls these endpoints with ids that belong to the caller, so the check would pass — but hard rule #3 says don't change what works, and this is explicitly ungraded. It is here so you can *explain* the gap if asked, not so you can close it.

While we are here: two more ungraded gaps in the clerk

- **No server-side input validation.** No `@Valid`, no `@NotNull`, no length limits. Everything the signup form enforces is enforceable only in the browser.
- **Email is not unique in the database.** There is no `unique = true` on the column and no duplicate check in `createUser`. Two accounts can share an email — and because `findByEmail` returns a single `Optional`, a later login against that email would error rather than pick one.

10 Checkpoint 7 — The vault

7

MySQL, BCrypt, and the one-way street

UserServiceImpl · model/User.java · application.properties

Passwords are shredded, not locked away

The crucial idea: the database does **not** store your password, encrypted or otherwise. It stores a *hash* — the output of a one-way function. A safe can be opened with the right key. A shredder cannot be run backwards at all, and that is the point.

```
// service/UserServiceImpl.java — on the way in
user.setPassword(passwordEncoder.encode(user.getPassword()));   // BCrypt
return userRepo.save(user);

// what lands in the row:
// "user123"  →  "$2a$10$N9qo8uLOickgx2ZMRZoMye1J/..."
```

So how does login work if the original can never be recovered? It doesn't recover it. It shreds the newly typed password the same way and compares the two piles of shreds:

```
passwordEncoder.matches("user123", storedHash)   // → true / false
```

BCrypt is designed for exactly this, with two properties that a plain hash like MD5 or SHA-256 lacks:

- **A built-in salt.** Every hash includes random bytes, so two people with the same password get completely different stored values. An attacker can't spot shared passwords, and precomputed "rainbow tables" are useless.
- **Deliberate slowness.** The `10` in `$2a$10$` is a work factor — 2^{10} rounds. It costs you a few milliseconds per login and costs an attacker trying billions of guesses an eternity. Raising it to 12 makes every guess four times more expensive.

> **WHAT THIS BUYS YOU**
>
> If somebody stole the entire `user` table tomorrow, they would not have a single password. They would have a pile of salted, deliberately slow hashes and a long, expensive road ahead — and the longer the original passwords, the more hopeless it is. This is the single most valuable security property in the whole project.

And the hash never leaves the building either

```
// model/User.java
@JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
private String password;
```

One annotation, two effects: the field is *read* when a registration or login request comes in, and *skipped* when anything is sent back out. So the response to `POST /api/register` carries no password, and every nested `user` object inside an inquiry or feedback response is scrubbed automatically — without anyone having to remember. Login goes further still and answers with a `LoginDTO` that has no password field at all, so there is nothing to leak even by accident.

The vault's own door

SETTING	VERDICT	WHY
Browser cannot reach MySQL	good	The only path to the data is through the clerk, so every request passes checkpoints 4 and 5 first
Queries built by Hibernate / JPA	good	Parameters are bound, never pasted into the SQL string — SQL injection has nowhere to land. There is no hand-written SQL anywhere in the project.
`spring.datasource.password=examly`	weak	Database credentials in plain text in a file that goes into git. Fine for an assessment; in production this is an environment variable or a secrets manager.
`ddl-auto=update`	weak	The app is allowed to alter its own schema at boot. Convenient; in production you would use migrations and a database account with no DDL rights.
`show-sql=true` + `BasicBinder=TRACE`	weak	Writes every query *and its parameter values* to the console — user data into log files.

All three weak rows are the same kind of thing: perfectly sensible choices for a development and grading environment, and all three are deployment concerns rather than code concerns. None affects the 24 tests, and hard rule #3 says leave them alone.

11 The life of a password

One picture that ties checkpoints 1, 4 and 7 together: what happens to the characters someone types, from keyboard to disk — and why they can never travel back.

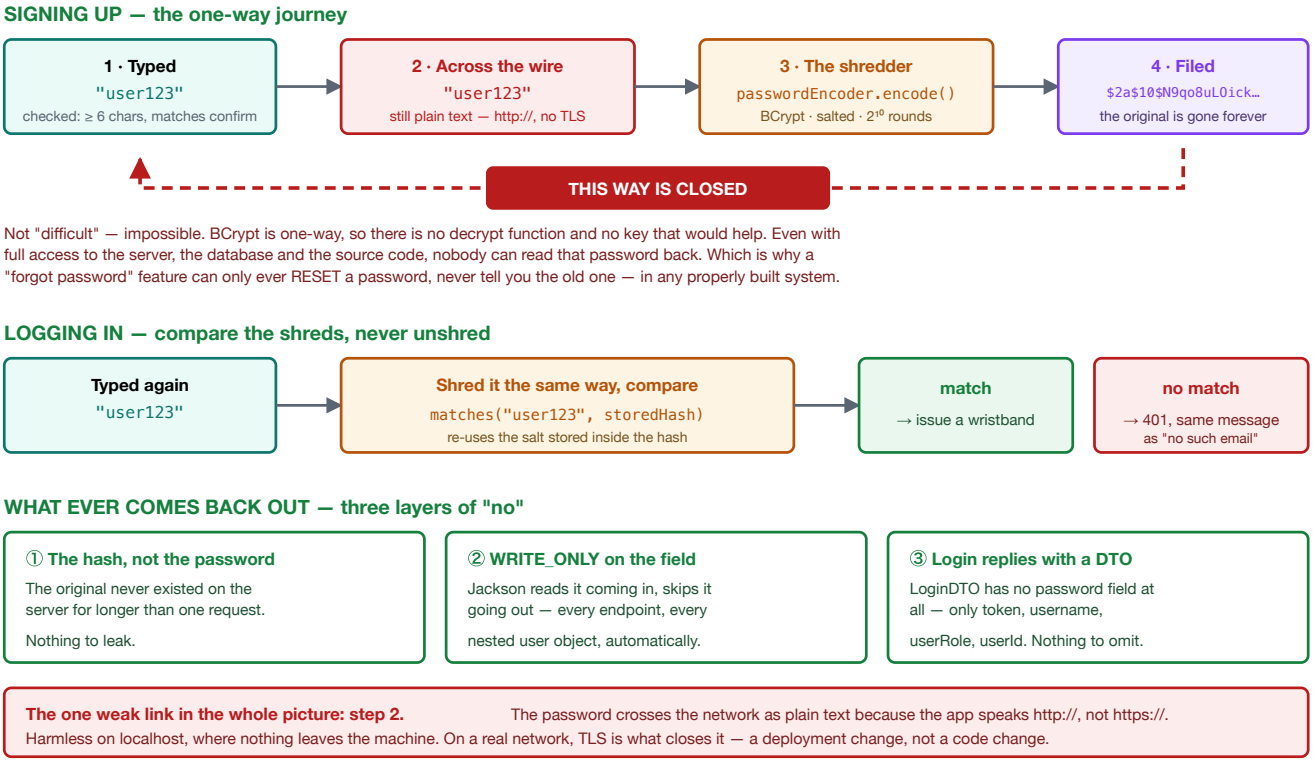


Figure 3 — A password's life. Three independent mechanisms make sure it never comes back out, and one missing mechanism (TLS) is why it is exposed on the way in.

12 Four attacks in slow motion

The best test of whether you understand a security model is to attack it. Here are four attempts in increasing order of sophistication — and exactly which checkpoint stops each.



Figure 4 — Four attempts, three dead ends and one open door. Note the pattern: the attacks that *look* clever are the ones the system handles; the one that succeeds is completely mundane.

A fifth, for completeness: stealing a token

Not drawn above because it needs something to go wrong first. If somebody obtained a valid token — by reading it off an unencrypted network, by getting a script onto the page, or by borrowing an unlocked laptop — they would simply *be* that user for up to 24 hours. There is no second factor, no device binding, and no way to revoke the token early. That is the accepted cost of a stateless design, and the reasons TLS, a shorter expiry and HttpOnly cookies exist as options.

13 The honest scorecard

What this project genuinely protects, and what it does not — stated plainly, so you can answer either question if you are asked.

Done properly

MECHANISM	VERDICT	WHY IT COUNTS
Passwords hashed with BCrypt	SOLID	Salted and deliberately slow. A stolen database yields no passwords.
Password never serialised out	SOLID	`WRITE_ONLY` covers every endpoint at once; login answers with a DTO that has no password field at all.
Signed tokens, verified every request	SOLID	Contents cannot be edited without the server-side secret.
Role read from the DB, not the token	SOLID	A role change takes effect on the very next request, without reissuing anything.
Permissions in exactly one file	SOLID	No `@PreAuthorize` competing with `SecurityConfig`. One place to read, one place to change.
Identical message for bad email and bad password	SOLID	Closes off account enumeration.
All SQL generated by JPA with bound parameters	SOLID	No hand-written SQL anywhere, so SQL injection has nowhere to land.
Stateless, no session fixation surface	SOLID	No session to hijack or fixate; CSRF disabled safely because auth rides in a header, not a cookie.

Open, ranked

GAP	SEVERITY	WHAT IT MEANS, AND WHY IT IS HERE
Anyone can register as an Admin signup form offers a Role dropdown; `createUser` saves `userRole` from the request body unchecked	HIGH	The registration endpoint is public and the role comes from the client. A stranger can create an Admin account and get full catalogue control. **Required by the design:** test BE-01 registers an Admin through this very endpoint, so it cannot be changed. In a real system, admins are seeded or promoted by an existing admin — never self-selected at signup.
No ownership check on `/user/{id}`	HIGH	Any logged-in User can read another user's inquiries or feedback by changing the number in the URL. Ungraded; explained in §9.
Plain HTTP, no TLS	MEDIUM	Passwords and tokens cross the network unencrypted. Harmless on `localhost`; a deployment concern, not a code one.
JWT secret hard-coded in the source	MEDIUM	Anyone with the repository can mint a valid Admin token for any email. Belongs in an environment variable.
No server-side input validation	MEDIUM	No `@Valid`, no length limits, no unique email. Every rule the signup form enforces can be skipped with `curl`.
Tokens cannot be revoked; 24-hour life	MEDIUM	Logging out is client-side only. A copied token keeps working until it expires. The standard stateless trade-off.
Database password in `application.properties`	LOW	Plain text in a file that goes into git.
`CORS allowedOrigins("*")`	LOW	Any website may call the API from a browser. Not a data risk by itself; narrow it in production.
SQL and parameters logged at TRACE	LOW	User data written to console and log files. Development convenience.
jjwt 0.9.1 (2018), deprecated API	LOW	An old library using the superseded `Jwts.parser().setSigningKey(String)` style. `pom.xml` must not be modified — hard rule #4.

HOW TO TALK ABOUT THIS LIST

Every open item is either required by the grading design (admin self-registration), a deployment concern rather than a code concern (TLS, secrets, logging, CORS), or explicitly outside the 24 tests (ownership checks, input validation). None of them is an accident of carelessness, and the project's hard rules say to leave all of them alone. Knowing *why* each one is there — and what you would do differently in production — is the useful part.

14 Coverage audit — nothing left behind

Every file in the repository that touches security, checked off against the section that explains it. If a file is not in this table, it has no security role.

Backend

FILE	§	ITS SECURITY ROLE
`config/SecurityConfig.java`	8	The door list. Every permission in the app; CSRF, stateless sessions, filter order.
`config/JwtAuthenticationFilter.java`	7	The bouncer. Validates the token and pins the identity on.
`config/JwtUtils.java`	6	Mints and verifies tokens; holds the secret and the 24-hour expiry.
`config/JwtAuthenticationEntryPoint.java`	7	Writes the 401 when no identity is present.
`config/MyUserDetailsService.java`	7	Turns an email into the live database row, on every request.
`config/UserPrinciple.java`	7	Builds the `ROLE_…` authority the door list matches against.
`config/CrosConfig.java`	5	CORS policy — which websites a browser may call the API from.
`service/UserServiceImpl.java`	10, 9	BCrypt hashing on register; constant-message credential check on login.
`controller/AuthController.java`	6, 10	Issues the token with the role from the DB row; replies with a password-free DTO.
`model/User.java`	10	`WRITE_ONLY` on the password — the field can be written, never read back.
`model/LoginDTO.java`	10	The login response shape; has no password field at all.
`exceptions/GlobalExceptionHandler.java`	9	One uniform refusal envelope; no detail leaks through it.
`exceptions/InvalidCredentialsException.java`	9	The 401 on a failed login — same message for both failure modes.
`exceptions/ResourceNotFoundException.java` + the other four	9	404 / 400 / 409 paths; no stack traces reach the client.
`service/*ServiceImpl.java` (the other three)	9	Shape validation and id resolution. No permission logic — by design.
`repository/*.java`	10	JPA-generated SQL with bound parameters; no hand-written queries.
`application.properties`	10, 5	DB credentials in plain text; `ddl-auto=update`; SQL + parameter logging.
`pom.xml`	13	Brings in `spring-boot-starter-security` and jjwt 0.9.1. Must not be modified.

Frontend

FILE	§	ITS SECURITY ROLE
`guards/auth.guard.ts`	4	The greyed-out map. Redirects by login state and role. Convenience, not a lock.
`app-routing.module.ts`	4	Carries `data: { role: … }` — the label the guard reads.
`services/auth.service.ts`	4, 6	The only writer of identity: stores the token and role; `logout()` clears them.
`services/architect.service.ts`	4	`authHeaders()` — attaches the `Bearer` token to every call.
`services/architect-inquery.service.ts`	4	Same. (Filename misspelling is a grading requirement, not a security matter.)
`services/feedback.service.ts`	4	Same.
`components/login/login.component.ts/.html`	4	Email pattern; distinguishes a 401 from a dead backend (`status 0`).
`components/signup/signup.component.ts/.html`	4, 13	Client-side rules — *and the Role dropdown* that makes admin self-registration possible.
`components/nav-base.ts`	4	Logout confirmation → `authService.logout()` → back to `/login` .
`components/feature-base.ts`	8	Translates 401 → "session expired" and 403 → "not authorized" for the user.
`app.component.html`	4	Picks the admin or user navbar from the stored role — cosmetic only.
`components/error/error.component.ts`	4	The landing place for guard refusals; sends people back to the right home.

The whole model in six sentences

1. The browser decides what to **show** you; the server decides what you can **do**. Only the second one is security.
2. Logging in trades a password for a sealed wristband; the password is shredded on arrival and can never be recovered.
3. Every later request carries that wristband, and the bouncer re-checks the seal and re-reads the person's real role from the database.
4. One file — `SecurityConfig` — then decides whether that role may open this particular door: **401** means "who are you?", **403** means "not you".
5. The two roles are not a ladder but two separate key rings that share exactly one door.
6. What is left open is either required by the assessment design, a deployment concern, or an ownership check nobody asks for — all three documented in §13, none of them to be changed.